

Tab 1



TRUSTLEVEL

ZK DeFi Protocol

Architecture Design Document

Proof of Achievement	
Proposal ID	1300196
Link	Project Catalyst Proposal
Milestone	1
Version	2.0 → Changes in: 3.2.1 → New: Appendix B
Status	Submitted and Approved - March 10th 2025
Contributors	Dominik Tilman
Report	Luozhu Zhang → Report
Milestone Acceptance Criteria	Architecture document must outline: 1. How zk-SNARKs will protect sensitive financial data and ensure private transactions. 2. Smart contract design must include zk-SNARK-based verification mechanisms for collateral and loan management. 3. Protocol needs to be modular and scalable and easy to implement Link: https://milestones.projectcatalyst.io/projects/1300196/milestones
Summary	This report outlines the architecture and system design for our zero-knowledge privacy protocol. The goal is to develop an open source ZK Proof-based protocol that ensures private, secure lending and borrowing, allowing users to verify collateral and execute transactions without revealing sensitive data. The design incorporates feedback from the community and establishes the foundation for a modular, scalable, and easily implementable protocol.

1. Ecosystem Review & Protocol Design Decisions

1.1 Ecosystem Survey Results

1.2 Comparison of ZKP Systems: PLONK, Groth16, and Halo2

- 1.3 Protocol Purpose and Capabilities
 - 1.4 Overview of User & Data Flow
- 2. Zero-Knowledge Layer
 - 2.1 Overview
 - 2.2 ZK Circuit
 - 2.3 Cryptographic Primitives
- 3. Smart Contract Layer
 - 3.1 Overview
 - 3.2 Smart Contract Components
 - 3.3 Integration of Smart Contracts and ZK Circuit
- 4. Off-Chain Layer
- 5. Integration Layer
 - 5.1 Overview
 - 5.2 Integration Components
 - 5.2 Integration Examples
 - 5.4 Technical Considerations
- 6. Implementation Considerations
 - 6.1 Implementation Steps for ZK Setup
 - 6.2 System Security & Threat Model
 - 6.3 Data Privacy & Compliance Considerations
 - 6.4 System Components Overview
 - 6.5 Future Enhancements & Roadmap
- Appendix
 - A. Trusted Setup for PLONK

1. Ecosystem Review & Protocol Design Decisions

1.1 Ecosystem Survey Results

1.1.1 Survey Methodology

We conducted a survey to gather community feedback on our ZK Protocol. We have shared the link to the Google form via direct messages on X and Discord to about 30 people that are active in DeFi on Cardano and got 12 responses.

- Survey link: <https://forms.gle/uG2g2kxsQkyKR5py6>
- Responses: [ZK DeFi Protocol on Cardano \(Responses\)](#)

1.1.2 Findings

Based on the survey responses, here are the most critical insights for your Protocol:

1. User Needs & Concerns:
 - **Top Privacy Priorities:** Users most want to protect their total holdings and the connection between their identity and transactions
 - **Current Barriers:** Most respondents dealing with institutional investors avoid DeFi due to concerns about exposing trading strategies, being targeted for liquidations, and general financial privacy
2. Integration & Cost Preferences:
 - **Platform Approach:** Most users prefer integration with existing platforms, though some see value in both integrated and standalone options
 - **Fee Tolerance:** 70% of respondents would accept higher fees for privacy, typically up to 25% higher
3. Use Cases:
 - **Institutional:** Protecting trading strategies, client fund management, and institutional operations requiring confidentiality
 - **Individual:** Personal security, over-collateralizing loans privately, and general financial privacy
4. Technical Concerns:
 - **Usability:** Worry about added complexity in the user experience
 - **Performance:** Questions about throughput and transaction times with ZK proofs
 - **Security:** Concerns about potential vulnerabilities in privacy implementations
5. Feature Requests:
 - **Privacy Controls:** Adjustable privacy tiers and selective disclosure options
 - **Advanced Features:** Private governance, anonymous lending pools, and integration with decentralized identity systems
6. Recommendations:

1. Design for integration with existing platforms while maintaining standalone capabilities
2. Implement tiered privacy levels to accommodate different user preferences
3. Focus on protecting collateral details and unlinking identity from transactions
4. Consider both PLONK and Halo2 as ZK setup.

1.2 Comparison of ZKP Systems: PLONK, Groth16, and Halo2

1.2.1 Introduction

Zero-knowledge proof systems are foundational to our protocol. These cryptographic methods allow one party (the prover) to convince another party (the verifier) that a statement is true without revealing any additional information beyond the validity of the statement itself. For our protocol, this means users can prove they meet collateral requirements without revealing their actual assets or balances.

Based on the feedback from our survey in chapter 1.1, we would like to use this chapter to compare three prominent zero-knowledge proof systems—Groth16, PLONK, and Halo2—to determine the most suitable choice for our DeFi protocol. We evaluate these systems based on setup requirements, performance characteristics, security properties, ecosystem compatibility, and current implementation status on Cardano.

Current Status of ZK Proofs on Cardano

Each of the three systems is being actively developed in Cardano. Below is a list of a few projects that we know about. Of particular interest is the recent successful implementation of Halo 2 on Cardano by the IOG research team (link to announcement: <https://x.com/InputOutputHK/status/1861422103292993683>).

- Modulo-P:
 - Groth16: <https://github.com/Modulo-P/Cardano-Semaphore/tree/main>
- ADAO
 - Groth16 & Plonk: <https://github.com/ADAOcommunity/zk/tree/main>
- zkFold:
 - Plonk: [zkFold: Cardano integration library](#)
- IOG:
 - Groth16: [Zswap: zk-SNARK Based Non-Interactive Multi-Asset Swaps](#)
 - Halo2: <https://github.com/input-output-hk/halo2>

Important note: Also, there is the Midnight protocol, a sidechain of Cardano for ZK proofs and ZK rollups. Utilizing Midnight for our protocol would of course simplify our development process. However, we clearly stated in our proposal that we will use Cardano as the blockchain for our protocol and do think that this is the reason why we got funded (e.g. Cardano Foundation voted for us for this reason). Therefore, we do not consider the integration of Midnight into our protocol as part of this particular proposal.

1.2.2 Overview of ZK Proof Systems

Groth16¹, developed by Jens Groth in 2016, is one of the most widely adopted and mature zk-SNARK constructions, with the following characteristics:

- **Trusted Setup:** Requires a circuit-specific trusted setup ceremony
- **Proof Size:** Very small (~200 bytes)
- **Verification Time:** Extremely fast verification (constant time)
- **Proving Time:** Highly efficient
- **Circuit Flexibility:** No circuit updateability; requires new trusted setup for any changes
- **Maturity:** Very mature with extensive production usage in systems like early implementations of Zcash

PLONK² (Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge) was developed by Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru in 2019, and is characterized by the following aspects:

- **Trusted Setup:** Universal and updatable trusted setup (one-time ceremony supports many different circuits)
- **Proof Size:** Moderately small (~600-800 bytes)
- **Verification Time:** Efficient, though slower than Groth16
- **Proving Time:** Reasonably efficient, slightly slower than Groth16
- **Circuit Flexibility:** Highly flexible, supports circuit updates without new trusted setup
- **Maturity:** Good maturity level, used in production systems like Aztec

Halo2³ is an evolution of the Halo proving system developed by Sean Bowe, Jack Grigg, and Daira Hopwood from the Electric Coin Company (creators of Zcash), with the following characteristics:

- **Trusted Setup:** Requires no trusted setup (uses recursion and accumulators)
- **Proof Size:** Larger than both Groth16 and PLONK
- **Verification Time:** Relatively efficient
- **Proving Time:** Less efficient than Groth16, comparable to PLONK
- **Circuit Flexibility:** Highly flexible with modular architecture
- **Maturity:** Newer system with ongoing development, recently deployed in Zcash

1.2.3 Detailed Comparison

1. Trusted Setup Requirements:

System	Setup Type	One-time or Per-circuit	Multi-party Ceremony	Risks
--------	------------	-------------------------	----------------------	-------

¹ <https://eprint.iacr.org/2016/260.pdf>

² <https://eprint.iacr.org/2019/953.pdf>

³ <https://eprint.iacr.org/2019/1021>

Groth16	Trusted setup	Per-circuit	Yes	If toxic waste from setup is not destroyed, proofs could be forged
PLONK	Universal trusted setup	One-time for many circuits	Yes	Lower risk due to universal setup, but still requires trust
Halo2	No trusted setup	N/A	N/A	No risks associated with trusted setup

→ Impact on Our Protocol: Halo2's trustless setup eliminates security risks and coordination complexity associated with trusted setup ceremonies, making it particularly attractive for a privacy-focused DeFi protocol where trust minimization is paramount.

2. Performance Characteristics:

System	Proof Size	Proving Time	Verification Time	Memory Requirements
Groth16	~200 bytes	Fast	Very fast (constant time)	Moderate
PLONK	~600-800 bytes	Moderate	Fast (but slower than Groth16)	Moderate-high
Halo2	~1-2 KB	Slower	Moderate	Higher

→Impact on Our Protocol: For DeFi applications on Cardano, verification time and proof size directly impact transaction costs and throughput. Groth16 offers the best performance in these aspects, which could lead to more efficient on-chain verification. However, the tradeoff is flexibility and setup requirements.

3. Security Properties:

System	Post-Quantum Security	Security Assumptions	Notable Security Reviews
Groth16	No	Knowledge of Exponent (KOE), Bilinear groups	Extensive academic review
PLONK	No	KOE, Bilinear groups	Substantial review
Halo2	Partial (uses inner product arguments)	Discrete Log	Newer, fewer reviews

→Impact on Our Protocol: While none of these systems are fully post-quantum secure, Halo2 makes fewer security assumptions. For a privacy protocol handling financial transactions, robust

security is essential. All three systems provide strong security under current cryptographic assumptions, but future upgrades may be needed for post-quantum resistance.

4. Circuit Flexibility and Development Experience:

System	Circuit Updateability	Developer Tooling	Domain-Specific Language Support	Learning Curve
Groth16	None (requires new setup)	Extensive	Good (many DSLs available)	Moderate
PLONK	High	Growing ecosystem	Good	Moderate-High
Halo2	High	Developing	Limited but growing	Steep

→ Impact on Our Protocol: As our DeFi protocol evolves, we'll need to update circuit logic to add features or fix issues. PLONK and Halo2 offer significant advantages here, allowing circuit updates without new trusted setups. Halo2's flexibility comes with a steeper learning curve, which could impact development timelines.

5. Cardano Compatibility:

System	Plutus Integration Status	Community Adoption	Existing Libraries	Technical Alignment
Groth16	Proof-of-concept	Growing	Few	Good
PLONK	Active development by zkFold	Growing	Few	Good
Halo2	Active development by IOG	Early stage	In development	Excellent

→ Impact on Our Protocol: IOG's active development of Halo2 integration with Plutus indicates strong ecosystem alignment for this approach. This backing from Cardano's core development team suggests better future support, optimization, and compatibility for Halo2-based solutions.

6. Scalability and Future-Proofing:

System	Recursive Proof Support	Aggregation Capabilities	Parallelization Potential
Groth16	Limited	Limited	Moderate
PLONK	Good	Good	Good
Halo2	Excellent (native support)	Excellent	Good

→ Impact on Our Protocol: As our DeFi protocol scales, the ability to handle complex operations efficiently becomes crucial. Halo2's native support for recursive proofs and aggregation presents significant advantages for future scaling, potentially allowing more complex privacy mechanisms as the protocol matures.

1.2.4 Analysis and Recommendations

Based on our comprehensive comparison, we assess the suitability of each system for our privacy-focused DeFi lending and borrowing protocol:

System	Strengths	Limitations
Groth16	• Exceptional verification performance • Smallest proof size • Mature implementation and tooling	• Circuit-specific trusted setup creates security and coordination challenges • Limited flexibility for protocol upgrades • Less alignment with Cardano's current ZK development focus
PLONK	• Good balance of performance and flexibility • Universal trusted setup simplifies protocol upgrades • Reasonable proof size and verification time • Growing ecosystem support (zkFold)	• Still requires trusted setup • Performance tradeoffs compared to Groth16
Halo2	• No trusted setup requirement aligns with DeFi security principles • Active integration with Plutus by IOG • Strong recursive proof capabilities for complex financial logic	• Larger proof sizes • Less mature than alternatives • Higher computational requirements • Steeper learning curve

Comparison Halo2 vs. PLONK

Feature	With Halo2	With PLONK
ZK Proofs	Merge circuits to reduce overhead	Merging circuits is even more beneficial
Nullifier Storage	Merkle tree-based	Use direct UTXO commitments (simpler and cheaper)
Collateral Verification	Stored in Merkle Tree	Store directly in UTXOs
Proving System	Slower, but strong recursion	Faster proving, but weaker recursion
Privacy Model	Pedersen commitments	Keep Pedersen, but consider Bulletproofs for nullifiers
Cardano Integration	Needs more development	Existing libraries available (zkFold, ADAO)

Recommendation:

For our Zero-Knowledge Privacy Protocol for DeFi lending and borrowing on Cardano, we consider **PLONK** as the primary zero-knowledge proof system. However, as a contingency plan, we will maintain awareness of **Halo2** developments.

Below are the justifications for PLONK::

- **Faster proving time** → PLONK has better performance than Halo2.
- **Smaller proof size** → ~600-800 bytes instead of ~1-2 KB.
- **Cheaper on-chain verification** → since Merkle tree updates are no longer needed.
- **Less state management** → No global Merkle tree for nullifiers, just UTXO commitments.

Below are the justifications for Halo2:

1. **Ecosystem Alignment:** IOG's active development of Halo2 integration with Plutus indicates strong future support and optimization.
2. **Trust Minimization:** The absence of trusted setup aligns with our goal of minimizing trust assumptions in the protocol.

1.3 Protocol Purpose and Capabilities

Purpose & Advantages:

The protocol should enable users to borrow assets and to provide liquidity (= lend assets) while maintaining privacy about their wallet (assets) or identity.

Advantages for borrower:

1. Confidentiality of the loan position
 - Loan amounts and collateral remain private
 - Competitors cannot see how much capital has been borrowed
 - Protection against profiling and monitoring
2. Protection against targeted liquidations
 - Prevents market manipulators from triggering targeted price movements to force liquidation
 - "Whale-hunting" (targeted attacks on large positions) is made more difficult
3. Collateral flexibility
 - Use of diversified collateral without public disclosure of the portfolio
4. Economic advantages
 - Avoidance of front-running for large positions
 - Better conditions by preventing price deviations due to own transactions
 - No disclosure of trading patterns or financial activity

Advantages for lenders:

1. Protection of the investment strategy
 - Hiding the allocation of capital to different lending pools

- Competitors cannot copy investment strategies
- 2. Asset protection
 - Reduced attack vectors as wallet size and capital are not publicly visible
 - Reduced risk of becoming the target of phishing or targeted attacks
- 3. More attractive platform
 - Investment product for institutional clients and HNWI (High Net Worth Individuals)
- 4. Separation of identity and capital
 - Participation in lending markets without disclosing personal financial information
 - Reduced risk for individuals in sensitive positions or from certain jurisdictions
- 5. Selective disclosure
 - Ability to selectively disclose data for tax or compliance purposes
 - Control over who has insight into financial activities

Capabilities:

The protocol should enable:

1. Private Collateralized Borrowing
 - Users can lock collateral and borrow against it
 - The specific assets used as collateral remain private
 - The collateralization ratio is privately verified but not publicly revealed
2. Anonymous Lending
 - Users can provide liquidity to lending pools
 - Their identity remains disconnected from their lending activity
 - The amount they've lent isn't publicly visible

Ecosystem Integration:

The primary value of our protocol would be as:

1. A Privacy Layer for Existing DeFi Protocols
 - Existing lending platforms (like Lenfi, Indigo, etc.) could integrate this as an optional privacy mode
 - Users could toggle between standard transparent operations and privacy-enhanced mode
 - The protocol would serve as middleware between users and existing DeFi platforms
2. A Modular Building Block for DeFi Applications
 - New DeFi applications could incorporate the privacy components as needed
 - Developers could import the privacy circuits into their own applications
 - The protocol could provide standardized privacy primitives for the ecosystem

Example Use Cases:

1. Private Collateralized Borrowing for Trading
 - Traders want to leverage their positions without revealing their strategy
 - Example: A trader holds a large amount of ADA and wants to borrow stablecoins to trade without signaling their intent or revealing their ADA holdings
 - The privacy protocol allows them to prove they have sufficient collateral without revealing which assets or how much
2. Institutional Lending with Confidentiality Requirements
 - Institutions need to participate in DeFi without exposing their positions

- Example: A fund wants to put capital to work in DeFi lending markets but can't publicly reveal its exposure or strategies
- The protocol allows the fund to provide liquidity while maintaining confidentiality
- 3. Privacy for High-Net-Worth Individuals
 - Individuals with significant crypto wealth need privacy for security reasons
 - Example: Someone wanting to borrow against their holdings without broadcasting their wealth
 - The protocol allows borrowing without creating an on-chain footprint that reveals wealth

1.4 Overview of User & Data Flow

This section provides an overview of the user and data flow of our protocol, which is then described and explained in detail in the following chapters. It starts with the Borrowing and Repayment flow, the Liquidity / Lending flow and finally shows how both flows are connected.

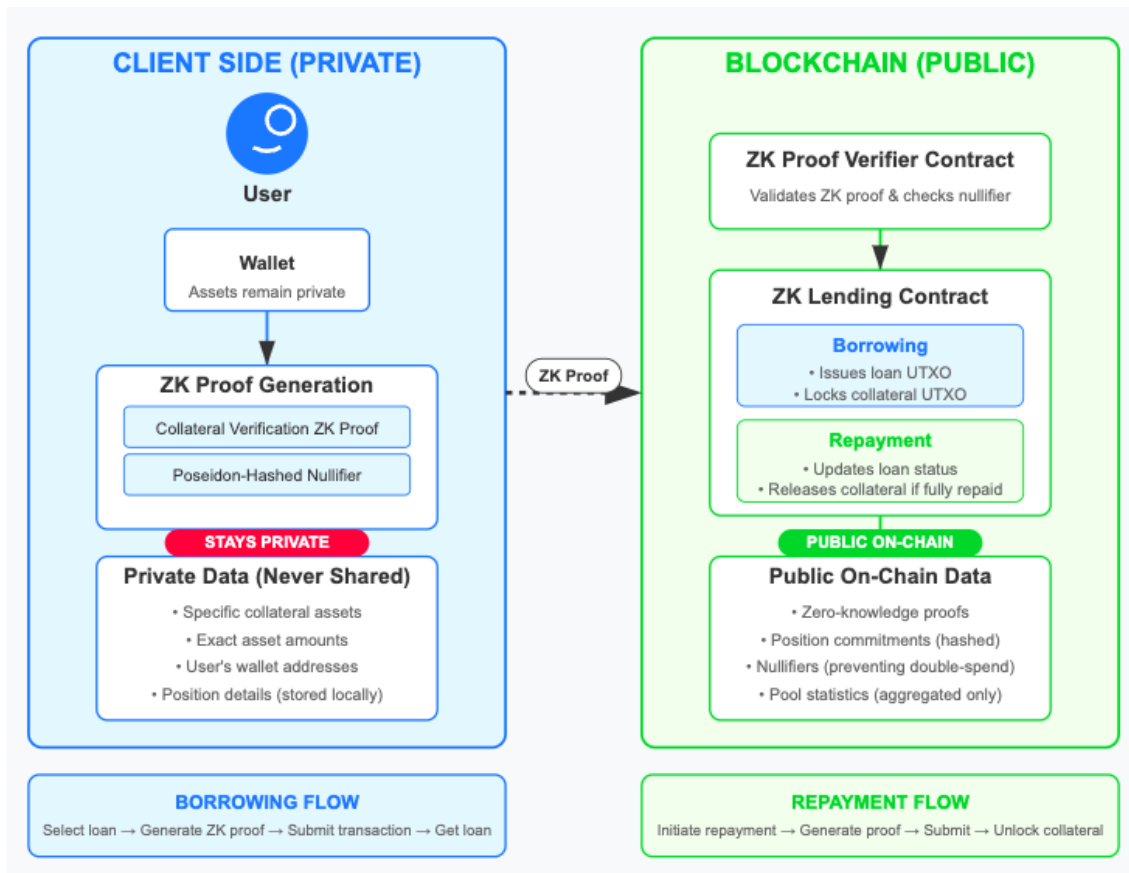
A. Borrowing Flow:

1. Collateral Verification & Loan Request:
 - User connects wallet and selects desired loan amount
2. Client application (off-chain):
 - Scans user's wallet for eligible collateral assets
 - Generates a Collateral Verification ZK proof
 - Creates a poseidon-hashed nullifier to the collateral (a hash-based representation)
3. User submit transaction that includes:
 - The ZK proof (proving collateral sufficiency without revealing exact assets)
 - Loan amount request
 - The nullifier commitment (to prevent reuse)
4. Smart contract verification
 - ZK Proof Verifier contract checks proof validity.
 - ZK Lending contract processes the loan request and issues a loan UTXO.
 - Collateral remains locked in a UTXO until repayment.

B. Loan Repayment Flow:

1. User initiates full or partial repayment
2. Client application:
 - Generates a Transaction Privacy ZK proof that authorizes the repayment
 - Creates a new nullifier to prevent double-spending of the repayment
3. User submits transaction with:
 - Repayment amount
 - Transaction Privacy ZK proof
 - Nullifier (ensuring UTXO uniqueness)
4. Smart contract verification:
 - ZK Proof Verifier checks proof validity.
 - If proof is valid:
 - Loan status is updated.

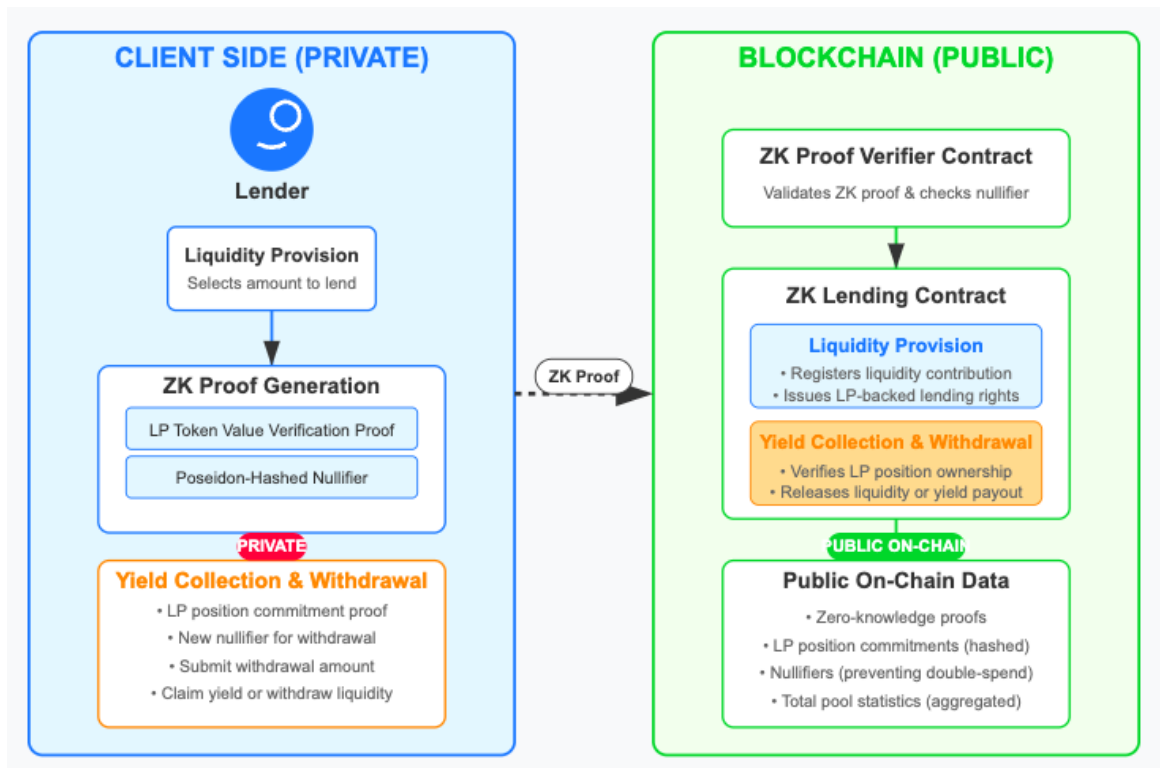
- If fully repaid, collateral UTXO is released.



C. Lending Flow

- Liquidity Provision:
 - User selects amount to contribute to lending pool
- Client application:
 - Generates a ZK proof verifying LP token value & liquidity depth.
 - Creates a Poseidon-hashed nullifier to prevent double-spending.
- User submits transaction with:
 - Deposit amount (LP tokens).
 - ZK proof of LP token validity.
- Smart contract verifies proof and adds liquidity to pool:
 - ZK Proof Verifier validates proof correctness.
 - ZK Lending Contract registers liquidity and issues LP-backed lending rights.
 - No public link exists between the lender's identity and their position.
- Yield Collection & Withdrawal
 - User wants to collect yield or withdraw liquidity
 - Client application:
 - Uses LP position commitment to generate a ZK proof.
 - Creates a new nullifier to ensure withdrawal cannot be replayed.
 - User submits transaction with:

- Yield claim or withdrawal amount.
- ZK proof of valid LP position ownership.
- Nullifier to prevent double withdrawals.
- Smart contract verifies proof, processes withdrawal/yield payment
 - ZK Proof Verifier checks the withdrawal proof.
 - ZK Lending Contract releases liquidity or yield payout.



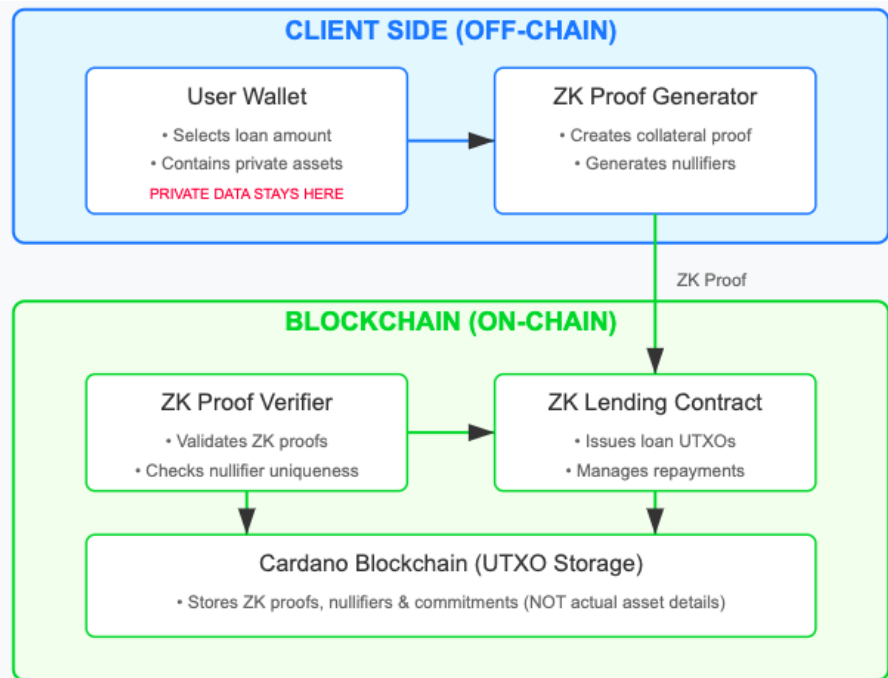
D. Data Flow

1. Private Data (Client-Side Only):

- Specific assets used as collateral and their amounts
- User's wallet addresses and private keys
- Loan position details (stored in encrypted local notes)
- Lending position details (stored in encrypted local notes)

2. Public On-Chain Data:

- Zero-knowledge proofs (Collateral Verification and Transaction Privacy)
- Commitments to positions (hash-based representations)
- Nullifiers (UTXO-based, prevent double-spending)
- Total pool statistics (aggregate amounts without individual breakdowns)



2. Zero-Knowledge Layer

2.1 Overview

Based on our analysis in Chapter 1, we have selected PLONK as a suitable zero-knowledge proof system. This section details how the system could be integrated into our protocol to enable private financial operations while maintaining the security and trust required for DeFi applications. This ZK layer will interface with the smart contract architecture described in the next chapter, which will implement the financial logic of the protocol while respecting the privacy guarantees established here.

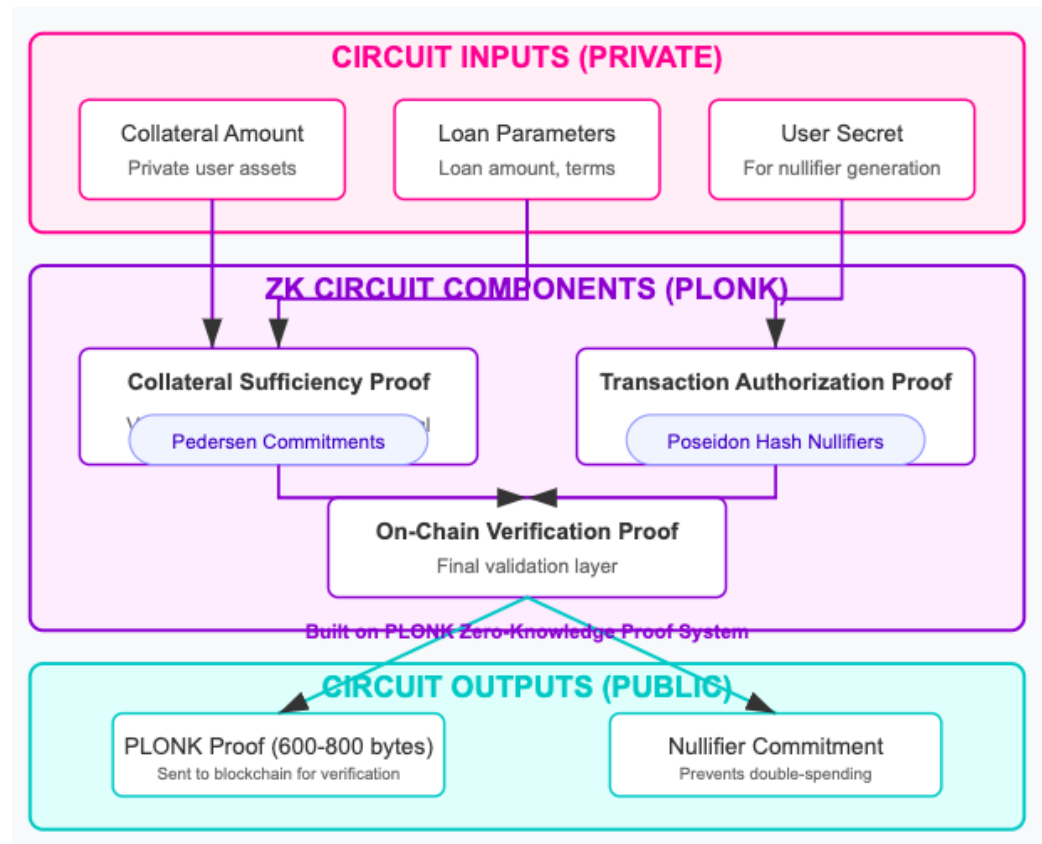
Our ZK system architecture provides the following privacy guarantees:

1. **Asset Confidentiality:** The specific assets used as collateral remain private
2. **Amount Confidentiality:** The exact values of collateral, loans, and repayments remain private
3. **Identity Protection:** Borrower identities cannot be linked to specific loans
4. **Relationship Privacy:** Connections between borrowers and lenders remain private
5. **Activity Concealment:** The timing and frequency of loan-related activities remain private

2.2 ZK Circuit

Circuits define precisely which statement is to be proven (e.g. “I have enough collateral”). Our ZK layer consists of a single circuit with three constraints, each responsible for different aspects of the lending and borrowing process.

2.2.1 Circuit Components



1. Collateral Sufficiency Proof (Private)

The Collateral Sufficiency Proof ensures that users hold enough collateral without revealing specific assets or amounts. This is achieved using Pedersen Commitments, which allow users to commit to a value without disclosing it.

Inputs:

- A commitment to the collateral amount (computed off-chain and included in the proof).
- The collateralization ratio required for borrowing.
- A proof that the user meets the required ratio.

How It Works:

1. The user provides a Pedersen Commitment for their collateral.
2. The circuit verifies that the committed collateral value meets or exceeds the required threshold.
3. The proof is generated off-chain and submitted along with the transaction.
4. The smart contract verifies the proof without revealing the actual collateral amount.

2. Transaction Authorization Proof (Private)

The Transaction Authorization Proof ensures that a user can perform valid borrowing, lending, and repayment transactions while preventing double-spending.

Inputs:

- The transaction type (borrow, repay, or lend).
- A nullifier commitment, which ensures the transaction is unique and cannot be replayed.
- The collateral proof output, linking the transaction to an existing valid collateral commitment.

How It Works:

1. The user provides a PLONK proof showing they are performing a valid transaction.
2. The proof includes a nullifier commitment to ensure the transaction is unique and prevent replay attacks.
3. The circuit verifies that the transaction follows protocol rules (e.g., loans must be over-collateralized, repayments reduce outstanding loans, etc.).
4. The proof is submitted to the smart contract for verification.

3. On-Chain Verification Proof (Public)

The On-Chain Verification Proof acts as the final validation layer, ensuring that all transactions processed through the protocol are correctly verified using PLONK proofs.

Inputs:

- The proof generated by the multi-purpose circuit.
- A public verification key (from the PLONK universal trusted setup).
- The nullifier commitments to check against existing transactions.

How It Works:

1. The smart contract receives the proof and public inputs.
2. It verifies the proof using existing PLONK verifiers.
3. If the proof is valid, the smart contract updates protocol state (e.g., issuing a loan, marking a repayment, or locking collateral).
4. If the proof is invalid, the transaction is rejected.

2.2.2. Implementation Details

1. How These Components Interact

The Collateral Sufficiency Proof, Transaction Authorization Proof, and On-Chain Verification Proof work together to ensure privacy-preserving lending and borrowing:

1. The user commits to their collateral via a private Pedersen Commitment.
2. The multi-purpose circuit generates a single proof that verifies:
 - The user has sufficient collateral.
 - The transaction is valid and prevents double-spending.
3. The proof is submitted on-chain, where the smart contract verifies it using PLONK verifiers.
4. If valid, the protocol state is updated (loan issued, repayment processed, etc.), without revealing sensitive financial data.

2. Mathematical Definition

PLONK proves a statement S is true without revealing anything else, using:

$$\pi = PLONK.Prove(S, w)$$

$$PLONK.Verify(\pi, S, vk)$$

- S : Public statement (e.g., “I have enough collateral”).
- w : Private witness (collateral amount, user’s secret values).
- π : The zero-knowledge proof itself.
- vk : The public verification key, used only when verifying the proof.

3. Benefits of This Design:

- PLONK proofs in this system are expected to be 600-800 bytes, allowing efficient on-chain verification.
- On-chain proof verification is expected to take 1-2 milliseconds, making it practical for DeFi applications.
- Single proof submission reduces transaction complexity and cost.
- Nullifier commitments prevent double-spending without requiring Merkle trees.
- On-chain verification is efficient and minimizes state updates.
- Fully compatible with Cardano’s eUTXO model and leverages existing PLONK verifiers.

2.3 Cryptographic Primitives

Our protocol relies on several cryptographic building blocks that work together to protect user privacy while maintaining system security. Think of these primitives as the fundamental tools in our privacy toolkit – each designed to solve a specific challenge.

In this chapter, we'll explain the two essential cryptographic primitives that form the foundation of our protocol:

1. **Commitment Schemes:** Allow users to "commit" to a value (like collateral amount) without revealing it, similar to a sealed envelope that can be opened later to verify its contents.
2. **Nullifiers:** Create one-time-use markers that prevent double-spending without revealing transaction details, functioning much like how a ticket is marked upon entry to prevent reuse.

2.3.1 Commitment Scheme

***Commitment Scheme** = "Like a sealed envelope containing your bid in an auction - others can't see what's inside, but you can't change it after submission"*

1. **Purpose:** Used for collateral sufficiency proof to ensure users can commit to a value without revealing it.

Why Pedersen?

- Hiding Property: The collateral value remains private due to the random blinding factor.
- Binding Property: Users cannot change their committed collateral value once submitted.
- Homomorphic Property: Supports addition operations, which is useful for batch verification.

2. Mathematical Definition:

A Pedersen Commitment to a value x (collateral amount) with a random blinding factor r is:

$$C = g^x h^r$$

where:

- g, h are generators of an elliptic curve group.
- r is a random scalar (blinding factor).
- x is the collateral value.

2.3.2 UTXO-based Nullifier Design

Nullifier = “Similar to how a ticket stub is marked after entry to prevent reuse”

Nullifiers are essential cryptographic primitives in every ZK protocol to prevent double-spending while maintaining transaction privacy. They create a one-time-use marker for private assets or operations without revealing which specific transaction or asset they're connected to.

1. Requirements:

- Each nullifier must be unique to prevent double-spending
- Only the legitimate owner can create valid nullifiers
- Observers cannot connect a nullifier to its originating transaction
- The same inputs must always produce the same nullifier
- The protocol can verify nullifier validity without compromising privacy

2. PLONK Approach:

- Use direct UTXO commitments instead of a global Merkle tree.
- Each transaction generates a Poseidon Hash-based nullifier commitment, uniquely identifying it.
- When a user repays a loan, they generate a PLONK proof that includes a nullifier commitment.

The smart contract does NOT store nullifiers but instead rejects transactions that try to re-use an already spent UTXO-based nullifier. This means the eUTXO model naturally prevents double-spending, rather than requiring a stored nullifier registry.

Note: The Cardano smart contract enforces nullifier uniqueness by ensuring that each UTXO contains a unique nullifier commitment. Once a UTXO is spent, it cannot be used again, preventing double-spending without requiring a global nullifier registry.

3. Poseidon Hash-Based Commitments

Why Poseidon?

- Optimized for zk-SNARKs → Reduces computation cost inside PLONK circuits.
- Collision-resistant → Ensures a unique nullifier for each transaction.
- Efficient field arithmetic → Works well with PLONK's native field operations.

4. Mathematical Definition:

A nullifier for a transaction T_i (e.g., a loan repayment) is computed as:

$$N_i = \text{Poseidon}(T_i, k)$$

where:

- T_i is the transaction commitment.
- k is a secret key derived from the user's private key.
- The output N_i is stored in a UTXO nullifier set to prevent reuse.

5. Structure:

Each nullifier will be computed as:

$$\text{Nullifier} = \text{Poseidon}(\text{UserSecret} \parallel \text{TransactionType} \parallel \text{Nonce})$$

Where:

- UserSecret: A secret known only to the user
- TransactionType: A code identifying the operation (borrow, repay, etc.)
- Nonce: A unique value that ensures different nullifiers for multiple actions on the same position

3. Smart Contract Layer

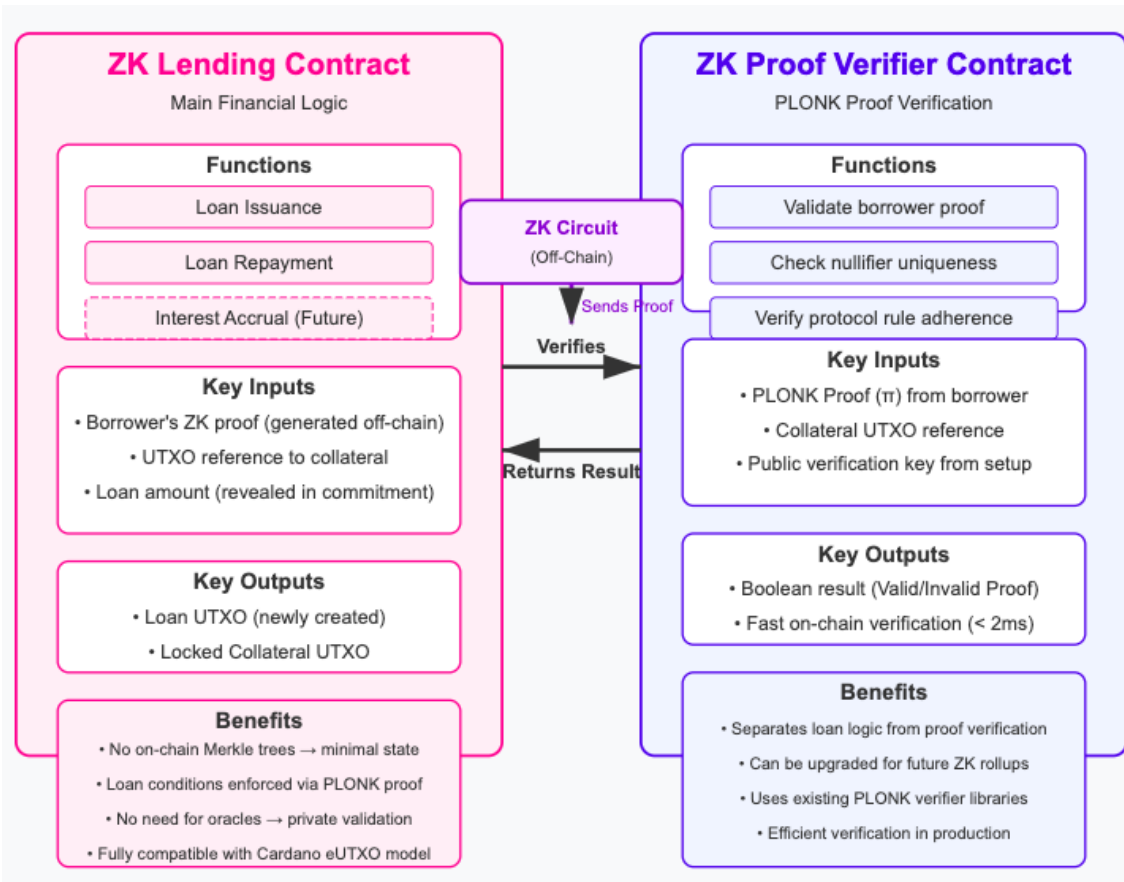
3.1 Overview

The smart contract layer of our protocol is responsible for enforcing protocol rules, managing financial state transitions, and verifying zero-knowledge proofs. This layer is designed to work seamlessly with the ZK system described in Chapter 2 while leveraging Cardano's extended UTXO (eUTXO) model and Plutus smart contract platform, which offers several advantages for our privacy protocol:

1. **State Isolation:** Each UTXO represents an isolated piece of state, allowing for parallel processing of loans.
2. **Concurrency:** Multiple users can interact with the protocol simultaneously without conflicts.
3. **Predictable Execution:** Transaction outcomes are deterministic, enhancing security.
4. **Privacy Alignment:** The UTXO model naturally supports privacy-preserving designs.

3.2 Smart Contract Components

Our ZK Circuit will interact with two smart contracts, one responsible for the financial logic of the lending and borrowing interaction, the other responsible for the integration of the ZK Proof verification.



3.2.1 ZK Lending Contract

Function: This is the main financial logic contract of the protocol → Handles loan issuance, repayments, interest rates and ensures valid borrowing rules. This smart contract could also be divided into different modules or even separate contracts if it's reasonable for the protocol's performance.

The ZK Lending Contract will manage:

1. Loan Issuance → Ensures the borrower has valid collateral and mints a loan UTXO.
2. Loan Repayment → Ensures that repayment transactions burn the correct loan UTXO.
3. Interest Payment → Interest rates and parameters stored in the loan UTXO metadata (for more details about the interest payment process see Appendix B).

Key Inputs

- Borrower's ZK proof (generated off-chain)
- UTXO reference to collateral (Pedersen Commitment)
- Loan amount (hidden in proof, revealed in commitment)
- Interest parameters (rate, compounding frequency, term)

Key Outputs

- Loan UTXO (Newly created after verification)
- Collateral UTXO remains locked (Cannot be spent unless repaid)
- Interest accrual verification result (when processing repayments)

Benefits of this Design:

- No on-chain Merkle trees, keeping state minimal.
- All loan conditions are enforced via PLONK proof verification.
- No need for oracles, since collateral is validated privately.
- Interest calculations don't compromise privacy guarantees.
- Flexible interest models can be expanded without major contract modifications.

3.2.2 ZK Proof Verifier Contract

Function: Verifies PLONK proofs on-chain and ensures borrowers have valid collateral and transactions are authorized

The ZK Proof Verifier Contract ensures that:

1. The borrower's proof is valid (they have enough collateral).
2. The nullifier is unique (no double-spending).
3. The proof adheres to protocol rules.

Key Inputs

- PLONK Proof (π) from the borrower
- Collateral UTXO reference
- Public verification key from PLONK setup

Key Outputs

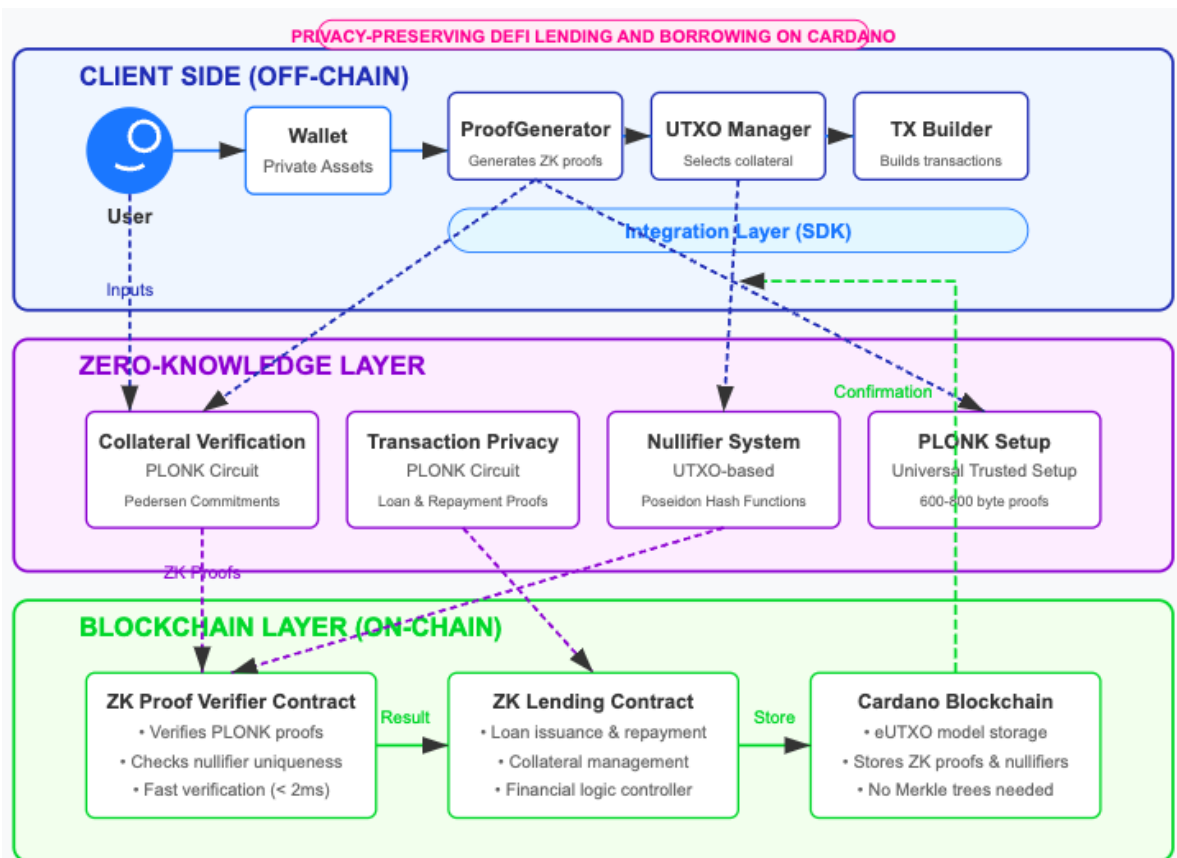
- Boolean result (Valid / Invalid Proof) → Used by the ZK Lending Contract
- Fast on-chain verification (< 2ms)

Benefits of this Design:

- Keeps loan logic separate from proof verification.
- Can be easily upgraded to support future ZK rollups.
- Builds upon existing PLONK verifier libraries for efficiency.

3.3 Integration of Smart Contracts and ZK Circuit

The ZK Circuit and Smart Contracts interact in a modular way to ensure privacy, correctness, and security while keeping the protocol efficient. Below is the workflow step-by-step described. The off-chain part is described in detail in chapter 4.



1. Borrower Creates Loan Request (Off-Chain)

- The borrower generates a PLONK proof using the ZK Circuit.
- This proof ensures:
 - Sufficient collateral is locked.
 - Loan amount follows protocol rules.
 - Nullifier is unique (prevents double-spending).
- The borrower submits:
 - PLONK proof ()
 - Transaction metadata (loan amount, collateral reference, etc.)
 - UTXO references (collateral commitment)

2. ZK Proof Verifier Contract (On-Chain)

1. Receives the PLONK proof from the borrower.
2. Checks the proof using zkFold's PLONK verifier.
3. Validates the nullifier to prevent replay attacks.
4. If proof is valid, it returns true to the lending contract.
5. If proof is invalid, the transaction is rejected.

3. ZK Lending Contract (On-Chain)

- Once the proof is verified, the ZK Lending Contract:
 1. Creates a Loan UTXO (locking funds for the borrower).
 2. Locks the collateral UTXO.
 3. Links the transaction to the borrower's address (without revealing identity).

4. Registers repayment rules (e.g., repayment UTXO structure).

4. Borrower Repays Loan (Off-Chain + On-Chain)

- When repaying, the borrower generates a new PLONK proof proving:
- The repayment matches the original loan terms.
- The correct UTXO is being spent.
- A new nullifier is generated (so repayment cannot be reused in another tx).
- The ZK Proof Verifier contract validates this proof.
- The ZK Lending Contract updates state:
- Marks the loan as repaid.
- Releases collateral back to the borrower.

Component	Role
ZK Circuit (Off-Chain)	Generates privacy-preserving proof of loan conditions (collateral, loan amount, nullifier).
ZK Proof Verifier (On-Chain, Plutus)	Verifies PLONK proofs and ensures the transaction is valid.
ZK Lending Contract (On-Chain, Aiken)	Handles loan issuance and repayments based on verified ZK proofs.
Borrower (Off-Chain)	Generates ZK proof and submits transactions.

Benefits of this Design:

- Smart contracts remain simple → Only loan logic and proof validation happen on-chain.
- Minimal state updates → UTXO model ensures privacy without Merkle trees.
- Efficient proof verification → zkFold PLONK verifier minimizes gas costs.
- Future-proof → Easily extendable for liquidation and interest calculations later.

4. Off-Chain Layer

1. Why is the Off-Chain Layer Important?

The Off-Chain Layer is a crucial component of our ZK-based lending protocol. While smart contracts enforce financial logic on-chain, many processes—including zero-knowledge proof (ZK) generation, UTXO selection, and transaction assembly—happen off-chain for efficiency and scalability. By offloading computationally expensive operations to the off-chain layer, we ensure:

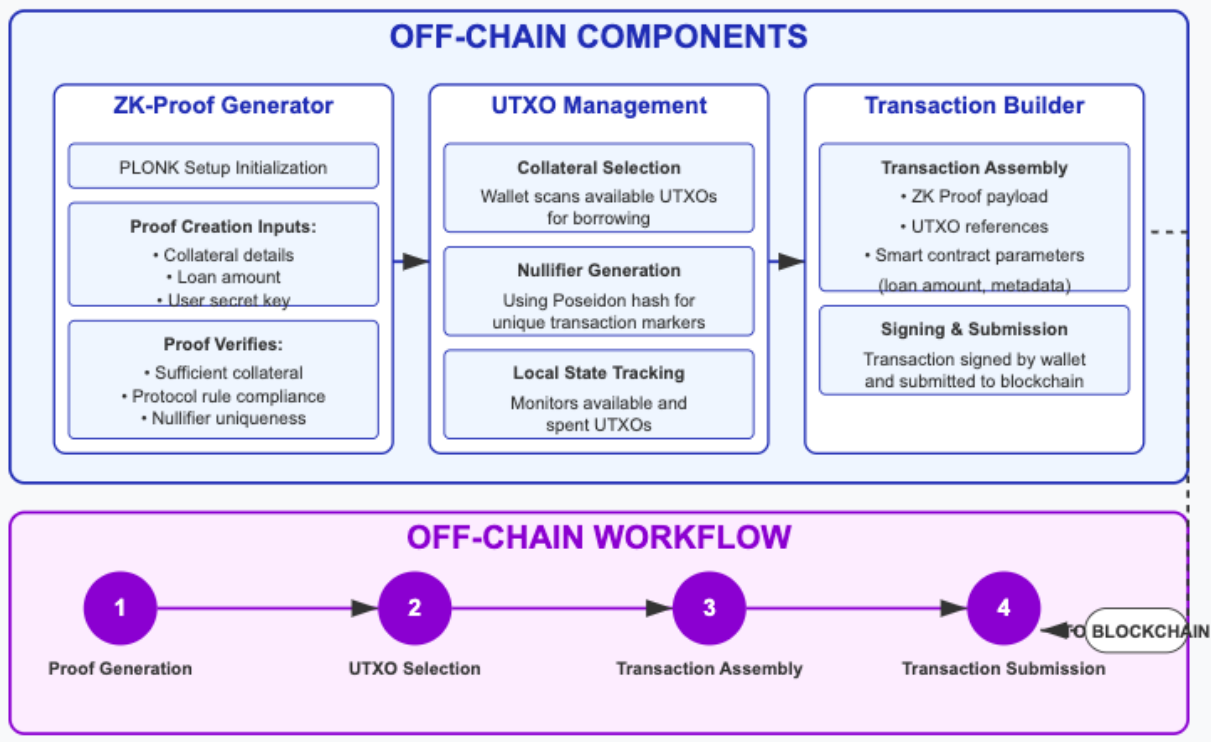
- **Cost Efficiency:** Proof generation and transaction assembly do not require gas fees.
- **Optimized Performance:** Smart contracts remain lightweight, focusing only on proof verification and state updates.
- **Enhanced Privacy:** User data and interactions remain off-chain, with only the necessary verifications happening on-chain.
- **Flexibility for Future Features:** Interest calculations, liquidation mechanisms, and governance can be integrated later without requiring major smart contract changes.

2. Off-Chain Workflow

The off-chain layer is responsible for generating proofs, managing UTXOs, and preparing transactions before submitting them to the blockchain.

Step	Description
1. Proof Generation	The borrower generates a PLONK proof for loan issuance or repayment.
2. UTXO Selection	The wallet selects the correct collateral UTXO and ensures nullifiers are unique.
3. Transaction Assembly	The borrower's wallet compiles the proof, UTXO references, and smart contract parameters.
4. Transaction Submission	The prepared transaction is signed and sent to the blockchain.

This process ensures that only **valid transactions** reach the blockchain, reducing the need for complex on-chain logic.



3. Key Off-Chain Components

- Off-Chain ZK-Proof Generator
 - PLONK Setup: Initializes public parameters needed for proof generation.
 - Proof Creation:
 - User inputs: collateral details, loan amount, user secret key.
 - The circuit generates a valid PLONK proof, ensuring:
 - The user has sufficient collateral.
 - The transaction follows protocol rules.
 - The nullifier is unique to prevent double-spending.
 - Wallet Integration: The proof is included in the transaction payload for submission.
- Off-Chain UTXO Management
 - Collateral Selection: The wallet scans available UTXOs to determine which one can be used for borrowing.
 - Nullifier Generation: Ensures that every transaction produces a unique nullifier using the Poseidon hash.
 - State Tracking: The wallet locally tracks which UTXOs are still available and which ones have been spent.
- Off-Chain Transaction Builder
 - Constructs the full transaction with:
 - ZK Proof (proving the validity of the loan or repayment)
 - UTXO References (collateral inputs, nullifiers)
 - Smart Contract Parameters (loan amount, borrower metadata)
 - The transaction is ****signed**** and submitted to the blockchain.

4. How the Off-Chain Layer Interacts with Smart Contracts

1. The borrower requests a loan and generates a ZK proof off-chain.
2. The borrower's wallet selects UTXOs and nullifiers.
3. The transaction is assembled and submitted to the ZK Proof Verifier contract.
4. If valid, the ZK Lending Contract issues a loan and locks collateral.
5. During repayment, the borrower generates a new ZK proof confirming the repayment details.
6. The repayment proof is submitted and, if verified, collateral is released.

5. Benefits of This Architecture

- Minimizes On-Chain Complexity: The blockchain only verifies proofs and updates states.
- Leverages Cardano's UTXO Model: No need for global registries or Merkle trees.
- Privacy-Preserving: Only necessary data is submitted on-chain, ensuring confidentiality.
- Future-Proof: Easily extendable for liquidation and interest accrual without major contract changes.

5. Integration Layer

5.1 Overview

Based on our survey (Chapter 1.1), we want to provide a lightweight SDK to facilitate seamless integration of our ZK Lending Protocol with existing DeFi platforms on Cardano. The SDK simplifies proof generation, transaction building, and smart contract interactions, allowing platforms to adopt privacy features without modifying their core infrastructure.

The privacy protocol can be integrated with host platforms through these architectural components:

1. ZK Adapter Layer:

- Translates between host platform data structures and ZK circuit inputs
- Implements standardized interfaces for common DeFi operations
- Manages proof generation and verification flows

2. State Management:

- Maintains privacy-preserving commitments alongside traditional state
- Provides mapping between private and public state when authorized

3. Integration SDKs:

- Client-side libraries for proof generation
- Server-side components for verification
- User interface components for privacy settings

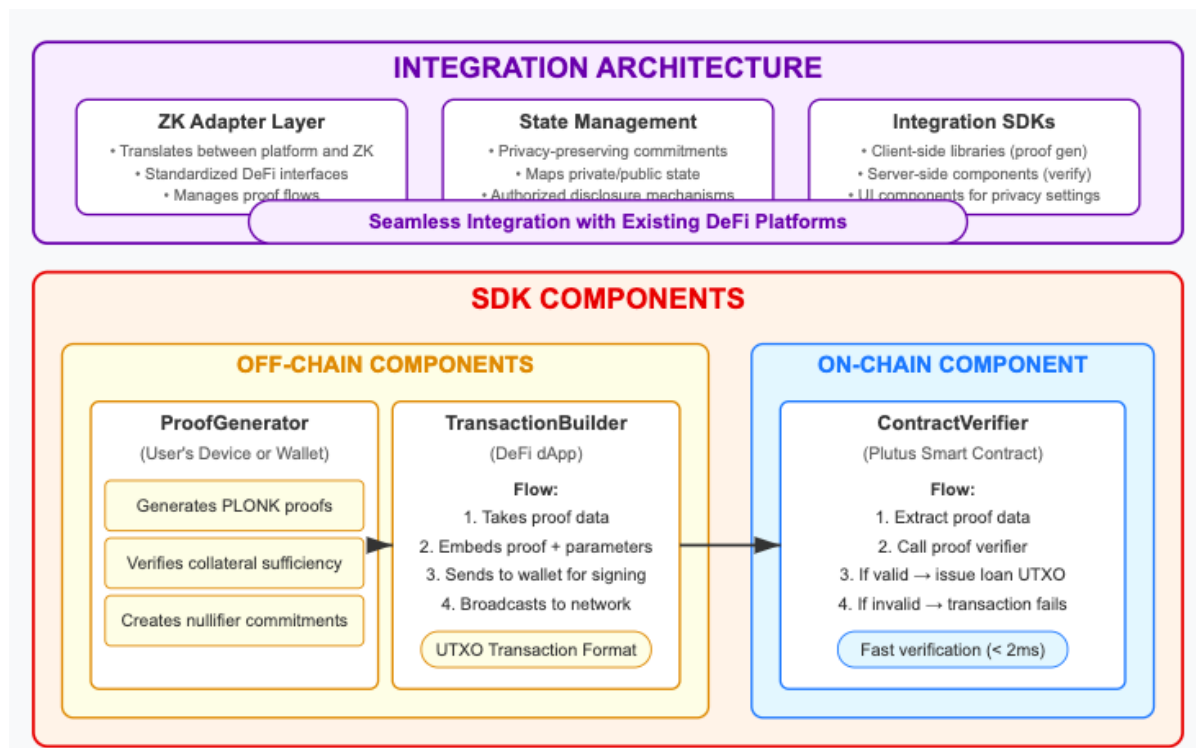
5.2 Integration Components

The SDK consists of three components:

Component	Function	Where it runs?
ProofGenerator	Generates ZK proofs for loans & repayments	Off-Chain (User's Device or Wallet)
TransactionBuilder	Creates UTXO-based transactions with proof data	Off-Chain (DeFi dApp)
ContractVerifier	Verifies submitted proofs in smart contract	On-Chain (Plutus Contract)

→ Why this setup?

- No complex state tracking → Proofs are stateless and embedded in transactions.
- No unnecessary abstraction layers → Works directly with existing wallets & smart contracts.
- Simple plug-and-play integration → Platforms only need to call basic functions.



A. ProofGenerator (Off-Chain)

1. What it does:
 - Generates a PLONK proof for loan issuance or repayment.
 - Ensures the borrower meets collateral requirements.
 - Uses Poseidon hashing for nullifier commitments.
2. How it works (Simplified Flow)
 - User selects collateral UTXO.

- The SDK generates a proof that:
 - The collateral is sufficient.
 - The nullifier is unique.
- 3. The proof is bundled into the transaction payload.

B. TransactionBuilder (Off-Chain)

1. What it does:
 - Constructs a Cardano UTXO transaction embedding the ZK proof.
 - Ensures the transaction is properly formatted for the smart contract.
2. How it works (Simplified Flow)
 - a. Takes proof data from ProofGenerator.
 - b. Embeds proof + loan parameters into the UTXO transaction.
 - c. Sends the transaction to the user's wallet for signing.
 - d. Broadcasts the signed transaction to the Cardano network.

C. ContractVerifier (On-Chain)

1. What it does:
 - Receives the transaction with ZK proof.
 - Verifies the proof via the Plutus smart contract.
 - If valid → Issues loan UTXO & locks collateral.
2. How it works (Simplified Flow)
 - a. The lending contract extracts proof data from the submitted transaction.
 - b. Calls the Plutus ZK proof verifier to check validity.
 - c. If valid, it locks collateral & issues a loan UTXO.
 - d. If invalid, the transaction fails.

5.2 Integration Examples

5.3.1 Lending Protocol

This example demonstrates integration with an existing lending protocol on Cardano, such as Liquid or Lenfi, to add privacy features without changing existing core functionality.

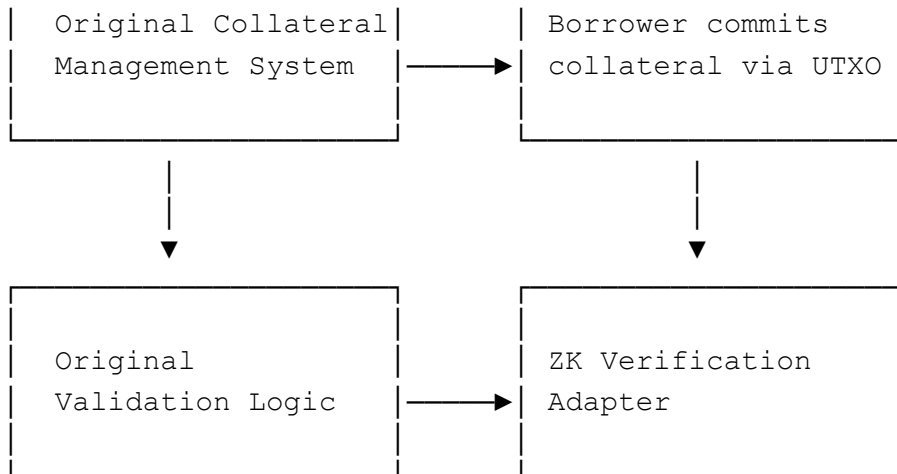
Typical components of lending protocol architecture:

1. **Liquidity Pools:** Smart contracts holding deposited assets for lending
2. **Interest Rate Models:** Algorithms determining borrowing and lending rates
3. **Collateral Management:** Systems for valuing, locking, and liquidating collateral
4. **User Interface:** Front-end for interacting with the protocol

The privacy layer integrates at these points:

1. **Collateral Privacy Integration:**





- Original platform's collateral system points to UTXO
- ZK adapter intercepts validation requests and performs ZK-based verification
- Collateral details remain private while maintaining security guarantees

2. User Flow:

Original:

1. User selects collateral assets.
2. Platform validates value and risk.
3. Loan is issued if collateral is sufficient.

New (ZK-based Flow):

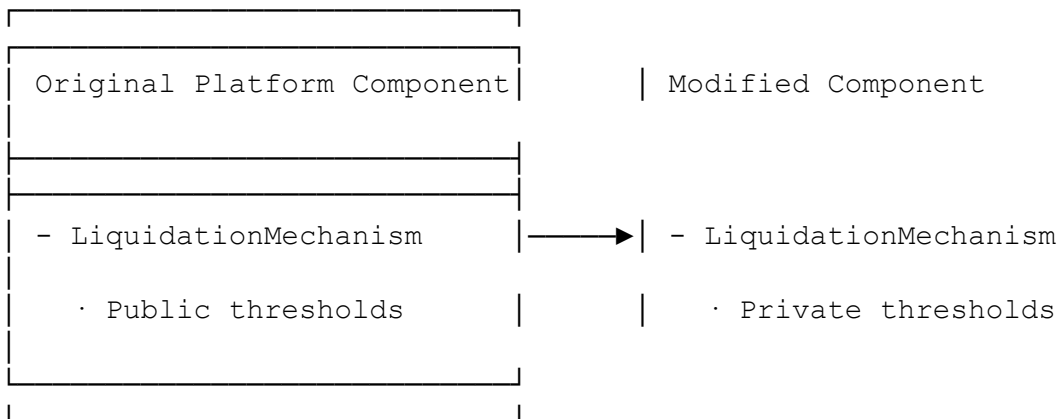
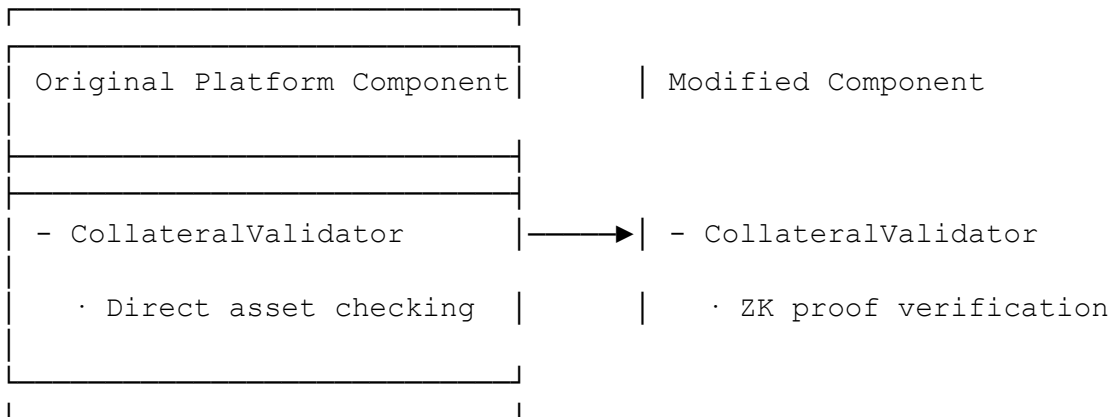
1. User selects collateral (off-chain, privately).
2. User generates a ZK proof that the collateral meets protocol conditions.
3. The ZK proof is submitted to the lending contract instead of revealing actual collateral details.
4. If proof verifies, the loan UTXO is created, and collateral remains locked.

3. Implementation Details

New Protocol Components:

Privacy Protocol Components
- ZkCollateralValidator - CommitmentScheme - ProofVerificationService - PrivacyEnhancedUtxos

Modified Existing Components:



Implementation Example in Haskell:

```
-- Original lending platform's collateral check (simplified)
validateCollateral :: PlatformParams -> Collateral -> LoanRequest -> Bool
validateCollateral params collateral loan =
  collateralValue collateral >= loanAmount loan * requiredRatio params

-- ZK-enhanced version
validateCollateralWithPrivacy :: LoanRequest -> ZkProof -> Bool
validateCollateralWithPrivacy loan proof =
  verifyZKProof proof (publicInputs loan)
```

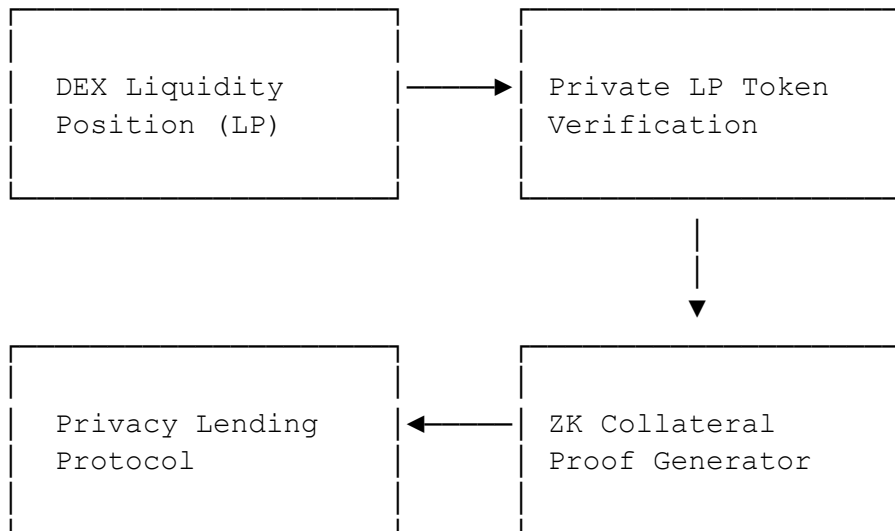
5.3.2 DEX Integration

This example shows how our privacy protocol can be integrated with a decentralized exchange on Cardano, such as Minswap or WingRiders.

Typical components of a DEX architecture:

1. **Automated Market Makers (AMMs):** Liquidity pools enabling token swaps
2. **Order Routing:** Systems directing trades to optimal pools
3. **Price Discovery:** Mechanisms determining exchange rates
4. **Liquidity Provision:** Systems for adding/removing liquidity

Integration points to enable privacy-preserving borrowing against LP positions:



1. LP Token Collateralization:

- Users can borrow against DEX LP tokens while keeping position details private
- ZK proofs verify the value and liquidity depth of positions without revealing exact composition

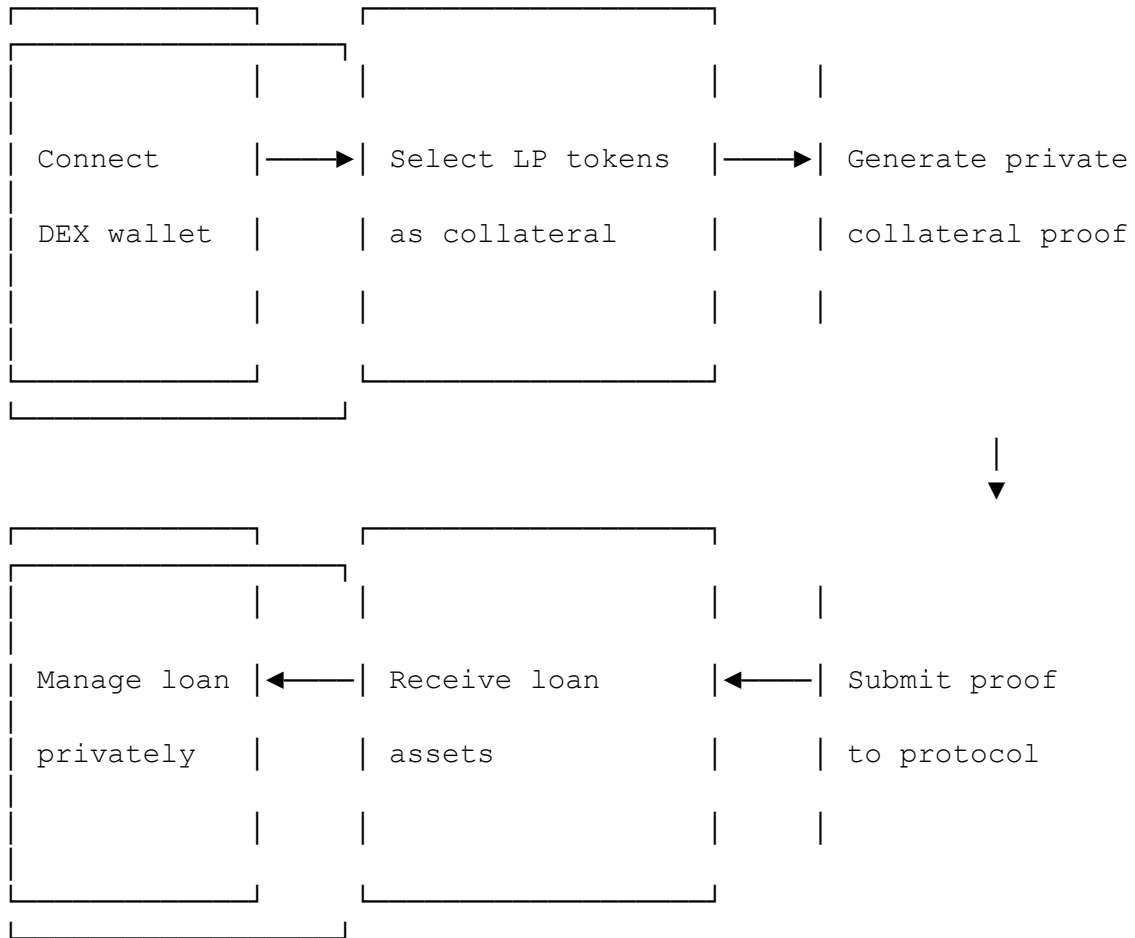
2. Implementation Example in Haskell:

```
-- DEX LP token validation in privacy protocol
validateLpCollateral :: DexParams -> LpTokenCommitment -> LoanRequest -> ZkProof -> Bool
validateLpCollateral dexParams commitment loan proof =
  -- Verify LP token value and risk characteristics without revealing pool position details
  verifyZkProof proof (publicInputs dexParams commitment loan)

-- LP token value calculation (performed in ZK circuit)
calculateLpValue :: LpToken -> OracleData -> Value
calculateLpValue lpToken oracleData =
  -- This calculation happens in the ZK circuit, not on-chain
  -- Only the result is proven, not the calculation itself
  let
    pool = lpToken.pool
    share = lpToken.share
    token1Value = pool.token1Amount * share * oracleData.token1Price
    token2Value = pool.token2Amount * share * oracleData.token2Price
  in
```

token1Value + token2Value

3. New User Flow:



Benefits for Users:

- For DEX Users:
 - Can leverage LP positions for loans without revealing trading strategy
 - Maintains privacy of pool allocation preferences
 - Protects against front-running when adjusting positions
- For DEX Platforms:
 - Adds new utility to LP tokens without platform modifications
 - Increases potential yield for liquidity providers
 - Attracts privacy-conscious users to the platform
- Prevents front-running of platform strategy allocations

5.4 Technical Considerations

5.4.1 Smart Contract Modifications

Example modifications to existing smart contracts for integration in Haskell:

```
-- Original validation function in Plutus
originalValidate :: LendingDatum -> LendingRedeemer -> ScriptContext -> Bool
originalValidate datum redeemer ctx =
  checkCollateral datum redeemer &&
  checkLoanParameters datum redeemer &&
  otherChecks datum redeemer ctx

-- Modified to support privacy features
enhancedValidate :: LendingDatum -> PrivacyEnabledRedeemer -> ScriptContext -> Bool
enhancedValidate datum redeemer ctx =
  case redeemer of
    StandardRedeemer r ->
      -- Original flow for users who don't need privacy
      originalValidate datum r ctx

    PrivacyRedeemer zkProof publicInputs ->
      -- Privacy flow using ZK proofs
      verifyCollateralProof zkProof publicInputs ctx &&
      verifyLoanParametersProof zkProof publicInputs ctx &&
      otherPrivacyChecks zkProof publicInputs ctx
```

5.4.2 Client-Side Integration

Example of client-side integration code in typescript:

```
// Integration with existing DeFi front-end
import { PrivacyProtocolSDK } from 'privacy-protocol-sdk';

// Initialize the SDK with host platform configuration
const privacySDK = new PrivacyProtocolSDK({
  platform: 'lending-platform',
  nodeUrl: 'https://cardano-node.example.com',
  circuitConfig: {
    collateralVerification: true,
    anonymousRepayment: true,
    privateInterest: true
  }
});

// Example of enhanced collateral selection UI
async function handleCollateralSelection(selectedAssets, loanRequest) {
  // Check if privacy mode is enabled
  if (userPreferences.privacyEnabled) {
    // Generate proof instead of revealing collateral details
    const proof = await privacySDK.generateCollateralProof({
      assets: selectedAssets,
      loanAmount: loanRequest.amount,
      loanCurrency: loanRequest.currency,
    });
  }
}
```

```
        requiredRatio: platformParams.requiredRatio
    });

    // Submit transaction with proof instead of asset details
    return submitPrivacyTransaction({
        proofData: proof,
        loanRequest: loanRequest,
        // No actual collateral details sent to network
    });
} else {
    // Original non-private flow
    return submitStandardTransaction({
        collateral: selectedAssets,
        loanRequest: loanRequest
    });
}
}
```

6. Implementation Considerations

6.1 Implementation Steps for ZK Setup

Implementation Step	Halo2	PLONK
Circuit Definition	Define a struct implementing <code>Circuit</code> trait with a <code>configure</code> and <code>synthesize</code> method	Define a circuit using constraint builder APIs
Constraint Building	Create custom chips and use a region-based allocation model	Use a selector-based gate model with wire assignments
Variable Management	Uses "advice columns" and "fixed columns" in a grid structure	Uses direct variable assignments to wires in gates
Custom Logic Implementation	Implement using the chip pattern, where each operation is a specialized <code>chip</code>	Implement using constraint selectors and custom gates
Lookup Tables	Define lookup tables explicitly in the circuit configuration	Typically uses gadgets or custom constraints instead of native lookups
Setup Phase	Generate verification key without trusted setup	Generate SRS (Structured Reference String) in trusted setup
Proving Process	1. Create prover struct 2. Assign values to advice columns 3. Generate proof using native recursion	1. Create witness 2. Assign values to variables 3. Generate proof using the SRS
Witness Generation	Assign values to advice columns via chip APIs	Directly assign values to variables
Testing Approach	Mock prover for circuit validation before full proving	Direct constraint satisfaction testing
Debugging Tools	Layouer debug tools to visualize constraint system	Constraint visualization tools
Integration with Plutus	Requires custom adapters for Plutus verification	Requires verification adapter for Plutus
Optimization Step	Focus on reducing row count and improving column structure	Focus on reducing gate count and selector efficiency
Final Deployment	1. Generate verification key 2. Create client-side proving library 3. Implement on-chain verification	1. Generate and distribute SRS 2. Create client-side proving library 3. Implement on-chain verification
Library Dependencies	<code>halo2_proofs</code> and related crates	<code>plonk</code> libraries (various implementations available)
Circuit Upgrades	Modify circuit and regenerate verification key	Modify circuit using same SRS

6.2 System Security & Threat Model

Threat	Potential Risk	Mitigation Strategy
Front-running	Users could manipulate transactions by front-running lending requests.	Using ZK-proofs ensures hidden transaction details, preventing front-running.
Double-Spending	A borrower might attempt to re-use a loan UTXO.	The nullifier system prevents double-spending, as UTXOs are uniquely linked to transactions.
ZK Proof Forgery	Malicious users may try to generate fake proofs.	The on-chain PLONK verifier ensures only valid proofs are accepted.
Key Leakage (Wallet Security)	If a user's private key is compromised, their assets may be stolen.	Users must secure their private keys, and future integrations could add MPC (Multi-Party Computation) wallets.

6.3 Data Privacy & Compliance Considerations

Privacy Features:

- **Asset Confidentiality:** Collateral amounts are committed via Pedersen Commitments.
- **Identity Privacy:** Borrowers are not linked to specific transactions.
- **Transaction Privacy:** UTXO-based model ensures that transactions do not reveal financial links.

Optional Compliance Considerations:

- While privacy is important, the system could support selective disclosure features in the future (e.g., zk-KYC compliance for regulated lending protocols).

6.4 System Components Overview

Component	Role	Technology Used
ZK Circuit	Generates privacy-preserving proofs	PLONK
ZK Proof Verifier (SC)	Validates proofs & nullifiers	Plutus or Aiken
ZK Lending Contract (SC)	Handles loan issuance & repayments	Aiken
Off-Chain Layer	Generates proofs, selects UTXOs, submits transactions	Rust or Python or Typescript
SDK - ProofGenerator	Generates ZK proofs for loans and repayments off-chain.	Rust or Python or Typescript
SDK - TransactionBuilder	Builds UTXO-based transactions embedding ZK proofs.	Rust or Python or Typescript
SDK - ContractVerifier	Calls Plutus contracts to validate proofs & trigger lending actions.	Rust or Python or Typescript

6.5 Future Enhancements & Roadmap

Feature	Why It's Important	Estimated Phase
Liquidation Module	Automates collateral liquidation if a borrower defaults.	After MVP
Advanced Interest Payment Module	Implementing advanced and flexible payment processing elements into the protocol.	After MVP
zk-KYC Integration	Optional feature for compliant lending markets.	TBD
MPC Wallet Support	Improves borrower security & decentralization.	TBD

Appendix

A. Trusted Setup for PLONK

Unlike older zk-SNARKs (such as Groth16) that require a new trusted setup for each circuit, PLONK uses a universal and reusable structured reference string. This makes it more efficient and adaptable for multiple circuits within our ZK Lending Protocol.

A trusted setup ceremony is a process where multiple participants contribute randomness to generate the Structured Reference String (SRS). This ensures that no single party has full control over the system, preventing backdoors that could allow proof forgery.

For our implementation, the trusted setup must:

- Support multiple circuits (collateral verification, loan issuance, repayments)
- Be performed transparently with multiple contributors.
- Allow updatable contributions (each participant adds randomness to prevent trust issues).

The ceremony consists of the following steps:

- Step 1: Choosing Participants
 - At least one honest participant is required to maintain security.
 - Multiple independent contributors are preferable (developers, community members, trusted organizations).
 - We are aiming for at least 5 participants for our PLONK setup.
- Step 2: Generating Randomness
 - Each participant generates a random secret value (τ) and applies it to the setup.
 - The value is not revealed to anyone, ensuring security.
- Step 3: Combining Contributions
 - Each participant applies their randomness to the existing structured reference string (SRS).
 - The final SRS is secure as long as one contributor destroys their secret randomness.
- Step 4: Verifying the Setup
 - The final output is publicly verified to ensure correctness.
 - The SRS is published for use in proof generation and verification.
- Step 5: Destroying Secret Values
 - Participants securely destroy their random values (τ) to prevent backdoor access.
 - If at least one participant destroys their value, the SRS remains secure.

After the trusted setup ceremony is complete:

- The final SRS is stored on-chain for smart contract proof verification.
- Our off-chain ProofGenerator SDK uses this SRS to generate valid PLONK proofs.
- The ZK Proof Verifier (Plutus contract) checks proofs against the SRS during loan issuance and repayments.

→ Example Trusted Setup from a Cardano Project:

- <https://github.com/Modulo-P/Cardano-Semaphore/tree/main>
- <https://www.youtube.com/watch?v=US3Tg3c5rK>

B. Interest Rate Payment Process

Implementing interest payments in a privacy-preserving lending protocol has some unique challenges. Traditional interest calculations are typically performed on-chain with full visibility

into loan amounts, durations, and rates. However, in a zero-knowledge protocol, these details must remain private while still ensuring correct interest calculation and payment.

This appendix details how the ZK DeFi Protocol implements privacy-preserving interest calculations, accrual, and payments while maintaining the core privacy guarantees of the system.

B1. Interest Model Architecture

The interest payment system consists of three main components:

1. **Interest Parameter Management:** How interest rates and terms are established and updated
2. **Interest Calculation Mechanism:** How interest is calculated while preserving privacy
3. **Interest Payment Processing:** How borrowers pay interest and how lenders receive it

Interest parameters are stored in two locations and are configured during loan origination:

1. **Public Protocol Parameters:**
 - Base interest rate ranges
 - Compounding frequency options (e.g., block-based, daily, etc.)
 - Maximum and minimum allowable rates
 - Interest model specification (e.g., fixed, variable, or hybrid)
2. **Private Loan-Specific Parameters** (stored in encrypted loan notes):
 - Specific interest rate applied to the loan
 - Loan origination timestamp
 - Interest accrual method
 - Rate adjustment terms (for variable-rate loans)

B.2 Interest Calculation

Interest is calculated using the standard compound interest formula:

$$FV = PV \times (1 + r)^t$$

Where:

- FV = Future Value (original loan amount + interest)
- PV = Present Value (original loan amount)
- r = Interest rate per period
- t = Number of periods

For privacy preservation, this calculation happens off-chain but is verified through zero-knowledge proofs (see B.8).

B.3 Off-Chain Calculation Process

1. The borrower's client interface retrieves:
 - Original loan amount (from local encrypted loan notes)
 - Interest rate (from loan parameters)
 - Loan duration/elapsed time (calculated from origination timestamp)
 - Compounding frequency (from loan parameters)
2. The client calculates:
 - Total periods elapsed (t)
 - Interest accrued using the compound interest formula
 - Total repayment amount (principal + interest)
3. The client generates a ZK proof that:
 - The interest calculation is correct based on the original loan parameters
 - The calculation follows the protocol's interest model rules
 - The total repayment amount is correctly derived from principal + interest

B.4 On-Chain Verification

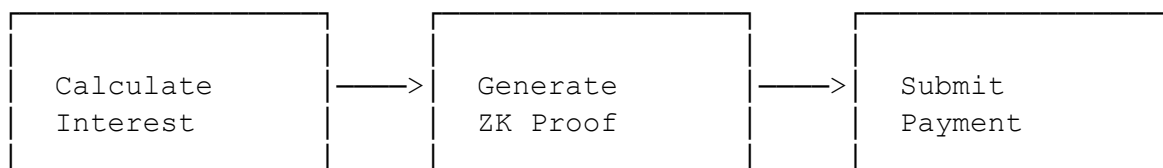
When processing an interest payment or loan repayment, the ZK Lending Contract:

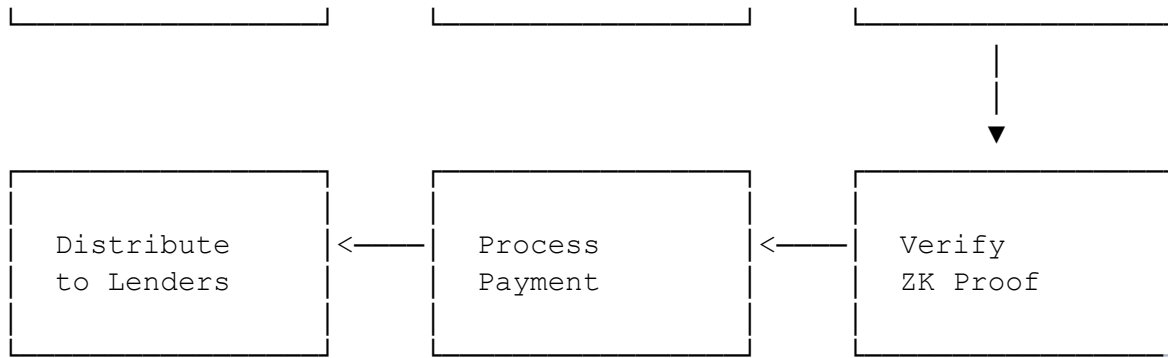
1. Receives the ZK proof of interest calculation
2. Verifies the proof without learning the actual loan amount, rate, or duration
3. Confirms the payment amount matches the proven interest calculation
4. Updates the loan state accordingly

B.5. Regular Interest Payments

For loans with periodic interest payments (separate from principal repayment):

1. **Payment Initiation:**
 - The borrower's client calculates accrued interest for the period
 - A ZK proof is generated demonstrating correct interest calculation
 - The borrower constructs a transaction with the interest payment and proof
2. **On-Chain Processing:**
 - The ZK Lending Contract verifies the interest payment proof
 - If valid, the interest payment UTXO is processed
 - The loan state is updated to reflect the payment
 - A new nullifier is generated to prevent double-payment
3. **Distribution to Lenders:**
 - Interest payments are added to the lending pool
 - Lenders can claim their share using privacy-preserving withdrawal ZK proofs





B.6 Full Loan Repayment with Interest

For loans where interest is paid along with principal at maturity:

1. **Repayment Calculation:**
 - The borrower's client calculates total accrued interest
 - A ZK proof is generated demonstrating:
 - Correct interest calculation
 - Proper identification of the original loan
 - Valid repayment amount (principal + interest)
2. **On-Chain Processing:**
 - The ZK Lending Contract verifies the repayment proof
 - If valid, the loan is marked as repaid
 - Collateral is released back to the borrower
 - Interest is distributed to the lending pool

B.7 Variable Interest Rate Implementation

For loans with variable interest rates, additional mechanisms are required:

1. **Interest Rate Updates:**
 - Rate adjustment events trigger updates to loan parameters
 - These updates are stored in protocol state with timestamps
 - The borrower's client tracks these updates for accurate calculations
2. **Segmented Interest Calculation:**
 - Interest is calculated in segments based on rate change timestamps
 - A ZK proof is generated that:
 - Verifies each segment's calculation is correct
 - Confirms proper application of rate changes
 - Demonstrates total interest is the sum of all segments
3. **Rate Change Verification:**
 - The ZK proof includes verification that rate changes follow protocol rules
 - This prevents manipulation of interest rates while maintaining privacy

B.8 ZK Circuit Additions

To support interest calculations, the protocol's ZK circuits must be extended to include:

1. **Interest Calculation Circuit:**
 - Inputs: Loan amount, rate, duration, compounding frequency
 - Outputs: Interest amount, total repayment amount
 - Constraints: Ensure calculations follow the specified formula
2. **Rate Verification Circuit** (for variable rates):
 - Inputs: Initial rate, rate change events, protocol parameters
 - Outputs: Proof that rate changes follow protocol rules
 - Constraints: Verify rate changes are within allowed bounds and follow the correct timing

B.9 State Management Enhancements

The following state management enhancements are required:

1. **Interest-Specific Nullifiers:**
 - Create unique nullifiers for interest payments to prevent double-payments
 - These are distinct from loan principal nullifiers
2. **Loan Status Tracking:**
 - Track interest payment history without revealing payment amounts
 - Maintain private running totals of interest paid vs. outstanding
3. **Timestamp Handling:**
 - Securely reference on-chain time for interest calculations
 - Use block numbers as time references to simplify implementation
 -

B.10 Interest Implementation Roadmap

Phase	Feature	Description
1 (MVP)	Basic Fixed-Rate Interest	Simple fixed-rate model with privacy-preserving calculation and validation
2 (MVP)	Variable Interest Rates	Support for rate changes with segmented calculation proofs
3	Advanced Payment Scheduling	Flexible payment schedules with privacy-preserving state tracking
4	Interest Marketplaces	Private bidding and rate discovery mechanisms
5	Cross-Asset Interest	Interest calculations involving multiple assets with complex relationships
